Tema 3.2:

# Arquitectura Transformer y funcionamiento de un LLM

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

# Tabla de contenidos


1 Introducción

2 El mecanismo de atención

3 Fase 1: *Encoding*

4 Fase 2: La capa Transformer

5 Fase 3: Modelado de lenguaje

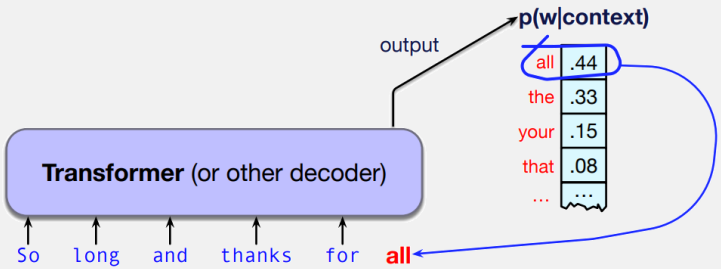
6 Operación

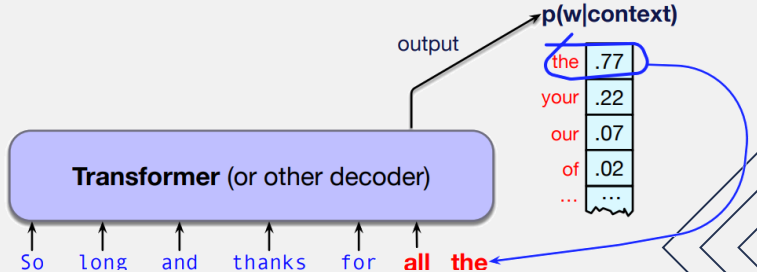
# 1

# Introducción

# Introducción: Generación autorregresiva (repaso)

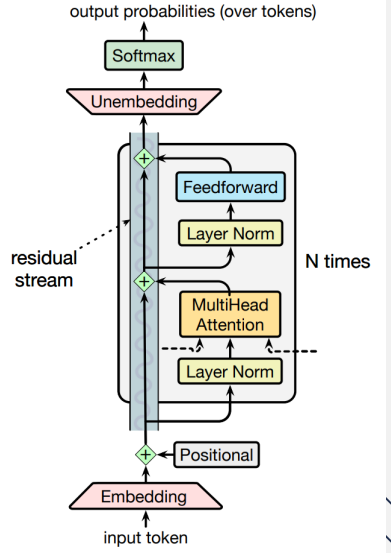
- Los LLMs generativos se basan en la **generación autorregresiva**: iterativamente, predicen el siguiente *token* y lo añaden al contexto.
  1. ***Tokenización***. Transformamos el contexto en *tokens* (generalmente, *subwords*).
  2. Calculamos la **distribución de probabilidades** del vocabulario para el siguiente *token*.
     - $P(w_i \mid w_{<i}) = P(w_i \mid w_1, w_2, \ldots, w_{i-1})$
  3. **Seleccionamos** el siguiente *token* (generación).
  4. **Añadimos el *token* al contexto** y repetimos el proceso iterativamente, hasta llegar al *token* EOS (*end of sequence*).



# Introducción: Etapas de procesamiento en la arquitectura Transformer (*decoder*)

- Dentro de la arquitectura Transformer *decoder*, podemos identificar 3 etapas:
  - **Codificación de la entrada (*input encoding*)**: Transforma el texto de entrada en un *embedding*, una lista de representaciones vectoriales sobre las que el modelo trabajará.
  - **Procesamiento de la información**: A través de bloques Transformer, el texto (en su representación vectorial) se procesa iterativamente, extrayendo las relaciones e información semántica de cada término y de su contexto.
  - **Modelado de lenguaje (*language modeling*)**: Se toma el resultado de la capa Transformer (*embedding*) y se procesa para predecir, en base a este vector, el siguiente término.



# Introducción: Las dimensiones de un modelo Transformer

- La capacidad de un modelo Transformer depende de su tamaño (número de parámetros), que queda delimitado por sus **hiperparámetros**. Los hiperparámetros más importantes son:
  - **Tamaño del vocabulario (V)**: Número de *tokens* únicos que el modelo reconoce.
  - **Dimensión oculta ($d_{model}$)**: Número de dimensiones de los *embeddings* que representan a cada *token* a lo largo de la arquitectura del modelo.
  - **Número de bloques ($N_{layers}$)**: Número de bloques Transformer que se encadenan uno tras otro.
  - **Ventana de contexto ($L_{max}$)**: Número máximo de *tokens* que el modelo es capaz de procesar durante una inferencia.

- Ejemplo:
  - El modelo Llama 3.1 8B tiene V = 128k *tokens*, $d_{model}$ = 4096 dimensiones, $N_{layers}$ = 32 capas Transformer y una ventana de contexto de $L_{max}$ = 8192 *tokens*.

# 2

# El mecanismo de atención

# Atención: Introducción

- Los términos, de manera aislada, poseen una semántica incompleta y ambigua.
  - Ej: En los *embeddings* estáticos (como Word2Vec), las palabras polisémicas se representan mediante un único vector que actúa como un promedio semántico de todos sus significados, perdiendo precisión.
- El **mecanismo de atención** resuelve este problema permitiendo que **la semántica fluya entre los *tokens* que pertenecen al mismo contexto**, logrando una representación contextualizada para cada una de sus instancias.
- Mecanismo de atención (alto nivel):
  - Para cada *token*, el modelo estima la **afinidad con el resto de *tokens*** del contexto. Esto es, el grado en el que cada *token* condiciona su significado.
  - Se realiza la **suma ponderada de la semántica** de cada uno de los *tokens* en el contexto del *token* en cuestión, **en base a la afinidad estimada** con cada uno de ellos.

# Atención: El mecanismo de atención

- La arquitectura Transformer utiliza **tres matrices de pesos ($W^Q$, $W^K$ y $W^V$)**, que aprende durante su entrenamiento, para proyectar la matriz de entrada X en tres espacios distintos:
  - ***Query* (Q)**: Representa lo que el *token* busca. $q_i = x_i W^Q$
  - ***Key* (K)**: Representa lo que el *token* ofrece para ser encontrado. $k_j = x_j W^K$
  - ***Value* (V)**: Contiene la carga semántica que se transferirá si hay afinidad. $v_j = x_j W^V$
  - Siendo $x_i$ el *embedding* de entrada del *token* i y $q_i$, $k_i$ y $v_i$ las proyecciones del *token* i en los espacios *query*, *key* y *value*.
- La **afinidad (*score*)** entre un *token* i y un *token* j se calcula mediante el producto escalar de la *query* del *token* a procesar ($q_i$) y la *key* del *token* en el contexto ($k_j$). Generalmente, normalizamos este valor por $\sqrt{d_{model}}$:

$$\text{score}(q_i, k_j) = \frac{q_i \cdot k_j}{\sqrt{d_{model}}}$$

# Atención: El mecanismo de atención

- La afinidad calculada se mide en logits ($\in [-\infty, +\infty]$). Utilizamos la **función softmax** para convertir estos valores en una **distribución normalizada** (la suma de todos los elementos es 1):
  - $\alpha_i = \text{softmax}_j(\text{score}(x_i, x_j))$
  - $\alpha_i$ es el **vector de peso de atención** del *token* i hacia el resto de *tokens* del contexto.
  - $\alpha_i(j) \neq \alpha_j(i)$ (el peso de atención del *token* i hacia el *token* j no es igual al peso de atención del *token* j hacia el *token* i)

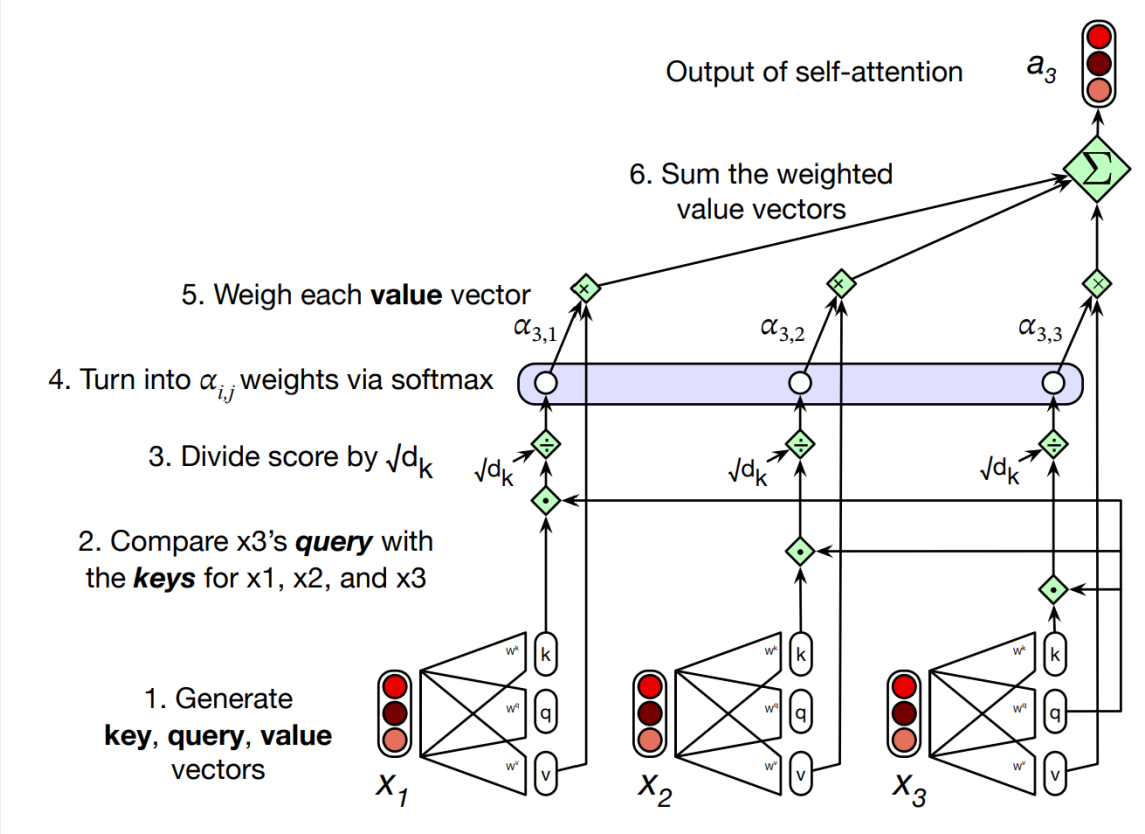
- El ***embedding* contextual** del *token* i se obtiene a través de la **suma ponderada del valor semántico** transferido por cada uno de los términos del contexto (los vectores *value* $v_j$), ponderado por el peso de atención entre cada par de términos:
  - $a_i = \sum_{\forall j} \alpha_{ij} \cdot v_j$

# Atención: El mecanismo de atención

- Durante el entrenamiento, en lugar de operar sobre cada *token* individual del contexto, calculamos estos valores en paralelo a través de **operaciones matriciales**.
- Sea X la matriz de los *tokens* de entrada del contexto $X = [x_0, x_1, \ldots x_n]$. Entonces:
  - $Q = XW^Q$.
  - $K = XW^K$.
  - $V = XW^V$.
- Donde Q, K y V son matrices que contienen los vectores de *query*, *key* y *value* de todos los *tokens* procesados.

- Calculamos la **afinidad** (logits): $S = \frac{Q \cdot K^T}{\sqrt{d_{model}}}$
- Calculamos los **pesos de atención**: $A$ = softmax($S$) (se aplica por filas).
- El resultado final (***embeddings* contextuales**) es: Attention(Q, K, V) = A · V.

# Atención: El mecanismo de atención



# Atención: El mecanismo de atención

- Ejemplo:
  - Contexto: “El banco paga intereses”. Supongamos $d_{model} = 64$.
  - *Tokenización* (supongamos que por palabras): [“el”, “banco”, “paga”, “intereses”]
  - Calculamos los *embeddings* estáticos de cada uno de los términos.
  - Ejecutamos el proceso de atención. Por ejemplo, para el término “banco”:
    - Obtenemos la *query* (q) de “banco”, $q_{banco}$.
    - Obtenemos las *keys* (k) de todos los términos del contexto.
    - Calculamos la afinidad (en logits) entre cada par como q·k.
      - q y k “reaccionan” (q·k toma un valor alto) cuando ambos términos comparten dominio semántico.
      - Normalizamos dividiendo los logits por $\sqrt{d_{model}}$.
    - Ej:
      - $q_{banco} \cdot k_{el} / \sqrt{d_{model}} = 1{,}2 / \sqrt{64} = 0{,}15$.
      - $q_{banco} \cdot k_{banco} / \sqrt{d_{model}} = 12{,}5 / \sqrt{64} = 1{,}56$.
      - $q_{banco} \cdot k_{paga} / \sqrt{d_{model}} = 25{,}4 / \sqrt{64} = 3{,}17$.
      - $q_{banco} \cdot k_{intereses} / \sqrt{d_{model}} = 30{,}1 / \sqrt{64} = 3{,}76$.

# Atención: El mecanismo de atención

- Ejemplo:
  - Ejecutamos el proceso de atención. Por ejemplo, para el término “banco” (2):
    - Convertimos estos valores en pesos de atención aplicando softmax:
      - softmax($q_{banco}$) = [0,01, 0,07, 0,35, 0,57].
      - Es decir, el *token* “banco” presta mucha atención a “intereses” (57%) y a “paga” (35%), más que a si mismo (7%), y apenas presta atención al *token* “el” (1%).
    - Calculamos, para cada término, la suma ponderada de la semántica que transmite cada término del contexto (v) ponderada por los pesos de atención:
      - $a_{banco} = 0{,}01\ v_{el} + 0{,}07\ v_{banco} + 0{,}35\ v_{paga} + 0{,}57\ v_{intereses}$
    - Como resultado del proceso de atención, el nuevo *embedding* contextual de “banco”, $a_{banco}$, ya no es un promedio de todos los posibles significados de esta palabra, si no que ha sido empujado al dominio financiero (92%), de forma que representa mejor su significado de “institución financiera”.

# Atención: Atención enmascarada en arquitecturas *decoder*

- Los modelos *decoder* (generativos) buscan **predecir el siguiente *token*** en base a sus predecesores:
  - Durante el **entrenamiento**, debemos **evitar que el modelo opere usando información del futuro**.
- Para evitar este problema, durante el entrenamiento, recurrimos a la **máscara de causalidad** (*causal mask*). La máscara de causalidad lleva a 0 los pesos de atención hacia *tokens* posteriores al que estamos procesando.
  - Justo **antes de aplicar softmax**, sumamos a la **matriz de afinidad (S)** una **matriz triangular** en la que las posiciones vinculadas a *tokens* del futuro llevan el **valor -∞** y el resto de posiciones un 0.
  - La función **softmax fuerza un peso de atención de 0** entre cada *token* del contexto y los *tokens* posteriores en el contexto.

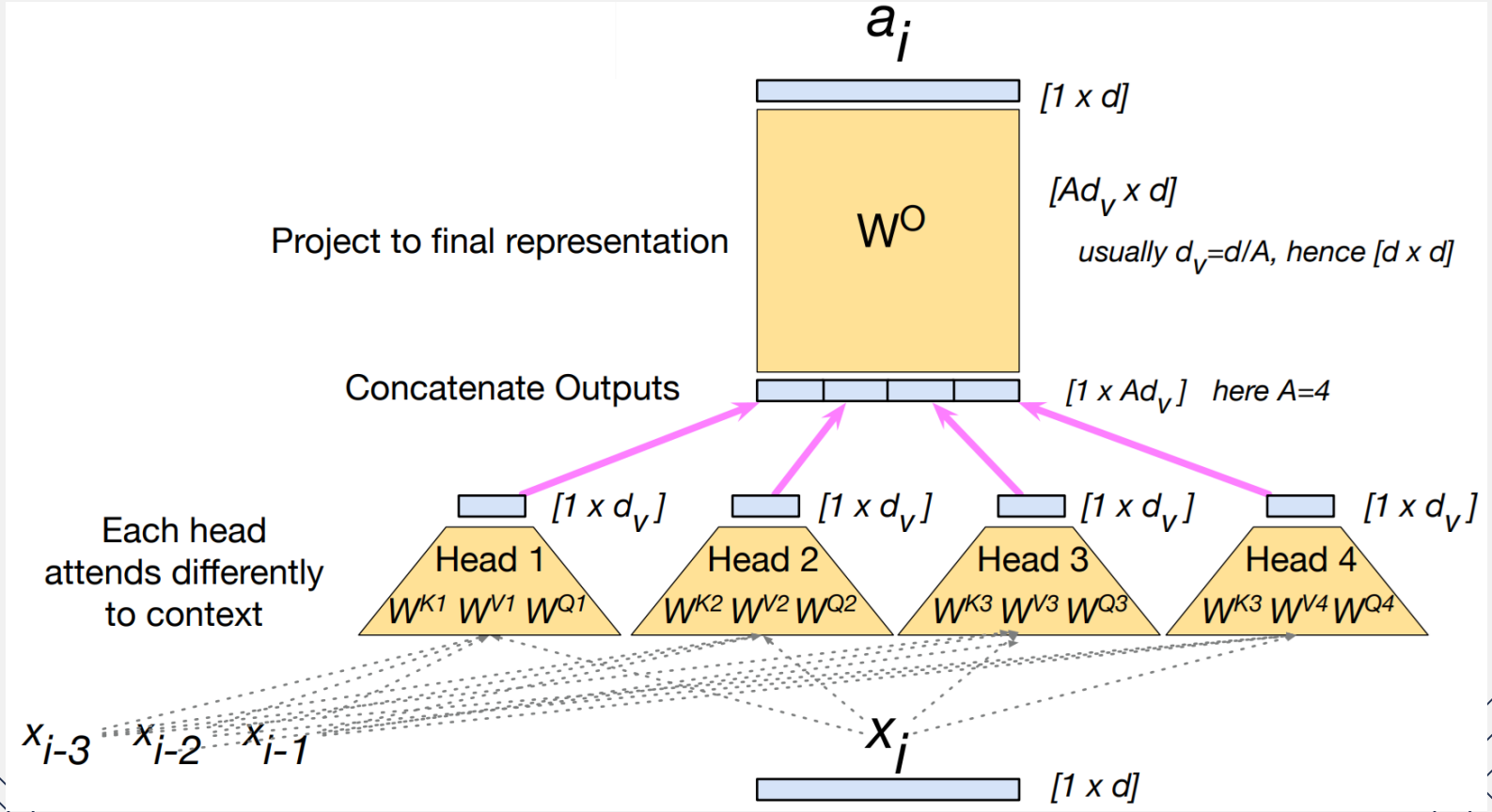
# Atención: Atención multicabeza

- Si tenemos una sola cabeza de atención, el modelo sólo puede enfocarse en un único tipo de relación (ej: relaciones sintácticas). Al dividir el *embedding* en **múltiples cabezas**, permitimos que el modelo **combine información de múltiples tipos de relaciones** (ej: sintáctica, gramatical, anafórica, temporal…).
  - Ejemplo: Dividimos las 4096 dimensiones en 32 cabezas de atención que procesan 128 dimensiones disjuntas.
- Para cada una de las cabezas de atención, **de manera independiente**:
  - Calculamos la afinidad: $S_k = \frac{Q_k \cdot {K_k}^T}{\sqrt{d_k}}$
    - Nota: **Normalizamos** por la raíz del **número de dimensiones de la cabeza ($\sqrt{d_k}$)**, en lugar del número de dimensiones del modelo.
  - Calculamos los pesos de atención: $A_k$ = softmax($S_k$) (se aplica por filas).
  - El resultado final para al cabeza *k* es: $\text{Attention}_k(Q_k, K_k, V_k) = A_k \cdot V_k$.

# Atención: Atención multicabeza

- Finalmente, se busca **distribuir la información de cada una de las cabezas específicas a lo largo del conjunto de dimensiones**.
  - Para ello, **concatenamos (horizontalmente) los vectores de atención** de cada una de las cabezas. Es decir, pasamos de *h* listas de vectores de tamaño $d_k$ a una única lista de vectores de tamaño $d_{model} = d_k \times h$.
  - Calculamos el **producto vectorial** de esta matriz de atención combinada, con una **matriz de pesos $W_o$, que distribuye la información especializada de cada cabeza entre todas las dimensiones** del *embedding* resultante.
- El resultado final son *embeddings* contextualizados donde cada *token* ahora contiene una representación rica, integrada y multidimensional, preparada para ser procesada por las siguientes capas del Transformer.

# Atención: Atención multicabeza



# Atención: Gestión de memoria en atención multicabeza

- Durante la inferencia, operamos sobre los valores *key* y *value* de *tokens* pasados. Para evitar calcular estos valores de nuevo, **se almacenan en memoria en las matrices K y V** (**caché KV**). Estas matrices almacenan tanto los valores de los *tokens* de entrada como los ya generados por el modelo.
- Si tenemos *h* cabezas, tendremos *h* matrices de pesos $W^Q$, $W^K$ y $W^V$, consumiendo mucha (V)RAM (principal cuello de botella).
- ***Multi-query attention*** **(MQA)**: Mantenemos *h* matrices de peso $W^Q$ para *querys*, pero entrenamos el modelo con una única matriz de *keys* $W^K$ y una única matriz de *values* $W^V$. El modelo se entrena para conseguir matrices $W^K$ y $W^V$ genéricas que representen suficientemente bien el conjunto de cabezas de atención. Reduce significativamente el consumo de memoria, aunque pierde precisión.
- ***Grouped-query attention*** **(GQA)**: El estándar actual. En lugar de utilizar un único par de matrices genéricas $W^K$ y $W^V$, utilizamos un conjunto reducido. Cada uno de estos pares $W^K$ y $W^V$ opera sobre un subconjunto disjunto de las cabezas de atención (ej: 32 cabezas, 8 matrices $W^K$ y $W^V$ para 4 cabezas).

# Atención: Gestión de memoria en modelos basados en atención

- Si tratamos de procesar un texto enorme (ej: un libro), para cada nuevo *token* a predecir, el mecanismo de atención opera sobre cada par de *tokens* anteriores ($O(N^2)$).
- ***Sliding window attention* (SWA)**: Se basa en que, en cada instante, los últimos W *tokens* son críticos y los N-W anteriores apenas aportan información relevante:
  - Se fija una ventana deslizante de tamaño W *tokens*. En lugar de operar con todos los *tokens* anteriores, sólo operamos con los W previos.
  - Se reduce la complejidad de $O(N^2)$ a $O(NxW)$.
  - Aunque en la primera capa de atención los *tokens* sólo acumulan información de los W *tokens* previos, la información fluye más allá del tamaño de la ventana conforme se encadenan bloques Transformer.
    - Ej: para W=4096, el *token* #6000 recibe información del *token* #2000 en la primera capa, y a su vez en la segunda capa el *token* #6000 transfiere información al *token* #10.000, incluyendo información de *tokens* pasados como el *token* #2000.

# 3

# Fase 1: *Encoding*

# F1-*Encoding*: Vectorización

- Esta fase tiene como objetivo **transformar el texto de entrada en un vector numérico**, que almacene la información semántica en un formato sobre el cual el modelo pueda trabajar.

- Comenzamos ***tokenizando*** la entrada (BPE, WordPiece…). Cada uno de los *tokens* se vectoriza a través de una matriz de *embeddings* estáticos.
  - No se utiliza un modelo pre-entrenado de referencia, si no que se utiliza una matriz propia $E$ ($V \times d_{model}$), que el modelo ajusta durante el pre-entrenamiento.
  - Como siempre, obtenemos el *embedding* indexando en la matriz por el ID de *token*.
  - Se trata de un *embedding* estático, que puede promediar múltiples significados.
- Problema: Si simplemente introducimos estos *embeddings* semánticos en los bloques Transformer, **el modelo no contempla el orden en el que ocurren los términos**:
  - "El perro muerde al hombre" genera las mismas activaciones que "El hombre muerde al perro".

# F1-*Encoding*: *Embeddings* posicionales

- Para solucionar este problema, recurrimos a los ***embeddings* posicionales**: introducimos un sesgo, en cada *token*, **en función de su posición en el contexto**.
- El modelo ajusta una **matriz de *embeddings* posicionales $E_{pos}$**, del mismo tamaño que E, que toma como entrada la posición en el contexto del *token* correspondiente.

- Tras obtener el *embedding* semántico y el *embedding* posicional de cada *token*, **ambos se suman, obteniendo un *embedding* compuesto (*composite embedding*)**:
  - No se concatenan, ya que duplicaríamos el número de dimensiones.
  - Dada la alta dimensionalidad, los vectores tienden a ser casi ortogonales. La suma de estos vectores suele ser simplemente un desplazamiento del *embedding* semántico, lo que permite recuperar ambas propiedades sin contaminar la información semántica original.

# F1–*Encoding*: *Embeddings* posicionales, RoPE

- Problema de los *embeddings* posicionales:
  - El número máximo de *tokens* que se pueden procesar queda acotado al definido en esta fase.
  - En general, **la posición absoluta no aporta mucha información** acerca de las relaciones entre términos.

- Solución: ***rotary position embedding*** **(RoPE)**:
  - En lugar de introducir información sobre la posición absoluta del *token* en el contexto, **operamos sobre las diferencias relativas entre cada *token***.
  - **No utilizamos *embeddings* posicionales**. En su lugar, **durante el mecanismo de atención, rotamos los vectores de *query* y *key*, de forma que la afinidad se ajusta en base a la distancia relativa entre los *tokens***.
  - Cada uno de estos vectores se rota nθ grados, siendo n la posición absoluta, de manera que la relación entre dos términos sólo depende de la distancia relativa.
    - La velocidad de rotación puede ser diferente para cada dimensión.
    - No es un parámetro entrenable, si no un hiperparámetro del sistema.
    - La velocidad de rotación es igual en todos los bloques Transformer.

4

# Fase 2: La capa Transformer

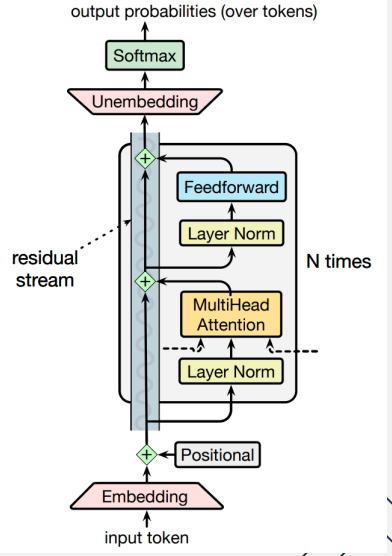
# F2-Transformer: La capa Transformer

- La capa Transformer está conformada por **$N_{layers}$ capas de bloques Transformer idénticos**, secuenciales.
- Cada bloque Transformer toma un conjunto de representaciones vectoriales (*embeddings*), lo procesan, y generan una nueva representación vectorial que combina la información de los *tokens* procesados.

- Cada bloque Transformer utiliza **dos mecanismos de procesamiento**:
  - **Autoatención multicabeza (multi-head self-attention)**: Basado en el mecanismo de atención, los *tokens* procesados comparten su información.
  - **Red neuronal *feedforward* (FFN)**: Procesa de manera aislada la representación existente correspondiente a cada *token* individual.
- Adicionalmente, antes de cada subcapa del bloque Transformer aplicamos **normalización, para estabilizar el entrenamiento**. Este paso reduce el ruido, evita la saturación y establece el rango numérico sobre el cual trabajará la siguiente subcapa.

# F2-Transformer: La capa Transformer

- Si procesamos los *embeddings* repetidamente (atención o FFN), **el modelo tiende a olvidar completamente la información original**, no es capaz de generalizar los cambios que debe aplicar sobre cada vector.
- Para evitar este problema, cada bloque Transformer utiliza un **flujo residual** (*residual stream*).
- En lugar de transformar completamente el *embedding* de entrada en cada procesado, **la transformación realizada por el bloque se suma al *embedding* original de entrada del bloque Transformer**.
- De esta manera, **el embedding nunca sufre cambios radicales**, simplemente es desplazado hacia una nueva dirección.
- Cada fase de procesamiento **no necesita aprender cómo transformar un vector de entrada en uno de salida, sino simplemente qué corrección debe aplicar sobre la entrada**, lo cual es más sencillo de aprender para el modelo.



# F2-Transformer: Procesamiento en el bloque Transformer

- Los *embeddings* se procesan en cada bloque Transformer de la siguiente manera:
  1. El ***embedding* de entrada original** se transporta **sin modificaciones** a través del **flujo residual**.
  2. Antes de entrar en el siguiente bloque de procesamiento, el *embedding* se **normaliza**.
  3. El *embedding* entra en la **subcapa de autoatención multicabeza**, donde recibe información contextual del resto de *tokens* en el contexto. El resultado de atención es el **desplazamiento que se debe aplicar** sobre el *embedding* de entrada. **Sumamos** ambos vectores.
  4. Antes de entrar en el siguiente bloque de procesamiento, el *embedding* se **normaliza**.
  5. El *embedding* entra en la **subcapa FFN**, que lo procesa de manera individual. El resultado de FFN es el **desplazamiento que se debe aplicar** sobre el *embedding* de entrada. **Sumamos** ambos vectores.
  6. El ***embedding* resultado** de esta suma es la **salida del bloque Transformer**.

# F2-Transformer: Procesamiento en la subcapa FFN

- La **FFN** se encarga de operar sobre el *embedding* tras el proceso de atención.
- El vector **ha recogido información del resto de *tokens*** en su contexto. La función del FFN es **ajustar este *embedding* para "colocarlo en una posición en el espacio" que refleje de manera más adecuada su significado, dado su contexto**.
- El proceso es como sigue:
  1. **Expansión**: El vector original **se proyecta a una dimensión mayor**, a través de su multiplicación por una matriz $W_1$ (ej: 4 x $d_{model}$). Las nuevas dimensiones son una **combinación lineal de las originales**. Trabajar sobre un espacio más amplio permite a la FFN aprender y dibujar relaciones complejas.
  2. **Activación**: Se aplica una **función no lineal** sobre los valores de cada dimensión.
  3. **Contracción**: El vector **se proyecta nuevamente a su dimensión original** ($d_{model}$), a través de su multiplicación por una matriz $W_2$ (combinación lineal de las dimensiones de la fase de activación).
- La expansión/contracción combina de manera lineal la información existente en diferentes combinaciones de dimensiones. Sin embargo, es necesario realizar una operación no lineal entre estas dos operaciones, que permita **revelar las relaciones relevantes latentes**.

# F2-Transformer: Procesamiento en la subcapa FFN

- Tipos de funciones de activación:
  - **ReLU (rectified linear unit)**: Lleva los valores negativos a 0 ($f(x) = \max(0, x)$).
  - **GeLU (gaussian error linear unit)**: Similar a ReLU, pero suavizado (se multiplica por la CDF de la distribución normal estándar).
  - **SwiGLU (swish gated linear unit)**: Cambia la arquitectura de la FFN. En lugar de utilizar un único proceso de expansión, se aplican dos:
    - Se construye una **matriz de contenido** multiplicando por una matriz $W_1$.
    - Se construye una **matriz de control** multiplicando por una matriz de expansión $W_2 \neq W_1$.
    - Aplicamos la función Swish a la matriz de control, obteniendo una **matriz que refleja la relevancia** de cada dimensión.
    - Escalamos los valores de la matriz de contenido mediante el **producto escalar con la matriz de relevancia**.
    - **Proyectamos** el resultado a las **dimensiones originales** mediante una matriz $W_3$.

# F2-Transformer: Normalización

- El propósito de la **normalización** de los *embeddings* antes de su procesado en las subcapas de atención y FFN es **reducir el ruido** y **evitar que los valores de sus dimensiones saturen**. También **establece el dominio de trabajo** (distribución numérica) sobre la cual trabajará la siguiente subcapa, de manera que sea más fácil conseguir que el modelo generalice las relaciones existentes.

- El estándar actual es **RMSNorm (*root mean square normalization*)**:
  - Tradicionalmente, se buscaba centrar los valores en 0 y escalarlos para obtener una varianza de 1 (LayerNorm, costoso).
  - RMSNorm simplemente **escala los valores en base a su magnitud absoluta** (más eficiente). No los centra en 0.

$$RMSNorm(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \cdot \varphi$$

  - (donde $\epsilon$ es un valor pequeño que evita posibles divisiones por 0, y $\varphi$ es un parámetro entrenable, que permite escalar de manera diferente los valores en cada capa).

# F2-Transformer: Mezcla de expertos (MoE)

- Modelos más “inteligentes” requieren arquitecturas más grandes. En un LLM, la FNN contiene el doble de parámetros que la subcapa de atención (computacionalmente más compleja).
  - Más parámetros -> **Mayor consumo de (V)RAM** y de cómputo.
- Para conseguir modelos más inteligentes sin aumentar la latencia de cómputo, recurrimos a la **mezcla de expertos (*mixture of experts*, MoE)**:
  - En lugar de construir una red FNN monolítica, construimos **N subredes FNN totalmente independientes (expertos)**.
  - Para cada *token* de entrada, **el modelo activa únicamente un subconjunto reducido de expertos**.
- Implicaciones:
  - Se reduce el número de operaciones en base al ratio de expertos que se activan, reduciendo significativamente la latencia durante la inferencia.

# F2-Transformer: Mezcla de expertos (MoE)

- **MoE requiere una modificación arquitectónica** dentro de las subcapas FFN:
  - Se añade un **router** a la entrada, que estima las **probabilidades de adecuación de cada experto** ante un *embedding* de entrada a la subcapa.
  - **Se divide la FNN en N subredes** independientes.

- Procedimiento:
  1. El **router evalúa el embedding** de entrada a la subcapa y genera una **distribución de probabilidades de adecuación a cada experto**.
  2. Se seleccionan los **K expertos con mayor probabilidad**.
  3. **Se envía el *embedding* únicamente a los K expertos más afines**.
  4. La **salida final** del sistema FNN MoE es la **suma poderada de los K resultados**, en base a la afinidad estimada por el router.

5

# Fase 3: Modelado de lenguaje

# F3-MdL: Generación del siguiente *token*

- El resultado de la capa Transformer (en *decoding*) es el *embedding* contextual del último *token* leído o generado. El *token* contiene la información del resto de palabras del contexto.
- Necesitamos **convertir ese *embedding* en una distribución de probabilidades, que nos permita estimar el siguiente *token***.
- El proceso de selección del *token* a generar es como sigue:
  1. **Proyección lineal de $d_{model}$ a V**: Obtenemos los logits para cada palabra del vocabulario del modelo.
  2. **Softmax de los logits**: Obtenemos la distribución de probabilidades del token a *generar*.
  3. **Selección del *token* a generar**.

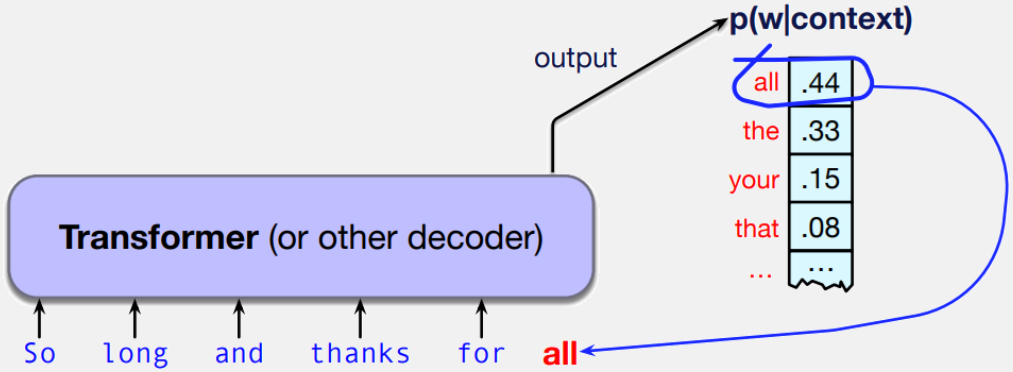
# F3-MdL: Cálculo de logits

- Los logits son valores $\in [-\infty, +\infty]$ que representan la **confianza del modelo a la hora de predecir el siguiente *token***.
- Los logits se calculan mediante una **proyección lineal del *embedding* de salida de la capa Transformer al tamaño del vocabulario**.
  - El modelo puede ajustar durante el entrenamiento una matriz de "unembedding"…
  - … o **utilizar la misma matriz E que usa en la fase de *encoding***.

- Antes de calcular la distribución de probabilidades para la estimación del siguiente *token* mediante softmax:
  - **Temperatura**: Se dividen los logits por un parámetro $\tau \in (0, +\infty)$, que regula la variabilidad a la hora de seleccionar el siguiente *token*.
  - **Filtrado top-k o top-p**: Limitan los elementos del vocabulario seleccionables, de manera que sólo se permite una selección sobre aquellos que permiten construir una frase con sentido.

# F3-MdL: Selección del siguiente *token* (repaso)

- Los modelos LLM procesan el contexto disponible, generando una **distribución de probabilidades** de la adecuación de cada *token* del vocabulario del modelo como aquél a predecir.
- Debemos **determinar qué *token* seleccionamos** en base a la distribución obtenida tras procesar el contexto:

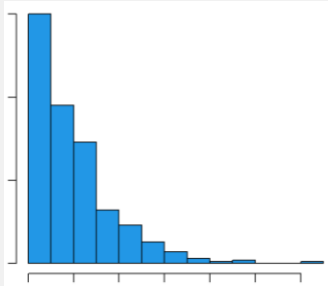


# F3-MdL: Selección del siguiente *token* (repaso)

- Los modelos LLM procesan el contexto disponible, generando una distribución de probabilidades de la adecuación de cada *token* del vocabulario del modelo como aquél a predecir.
- Debemos determinar qué *token* seleccionamos en base a la distribución obtenida tras procesar el contexto:
  - **Decodificación avariciosa (*greedy*)**: Seleccionar sistemáticamente el *token* que tiene la probabilidad más alta.
    - Ventaja: El resultado es (generalmente) coherente.
    - Problema: Los resultados suenan muy genéricos, planos y repetitivos.
    - 100% determinista, un mismo *prompt* genera siempre la misma respuesta.
  - **Decodificación por muestreo (*sampling*)**: Realizamos una selección aleatoria en base a la probabilidad de cada término.
    - Ventaja: El texto parece más humano, es más diverso y creativo.

# F3-MdL: El problema del muestreo aleatorio (repaso)

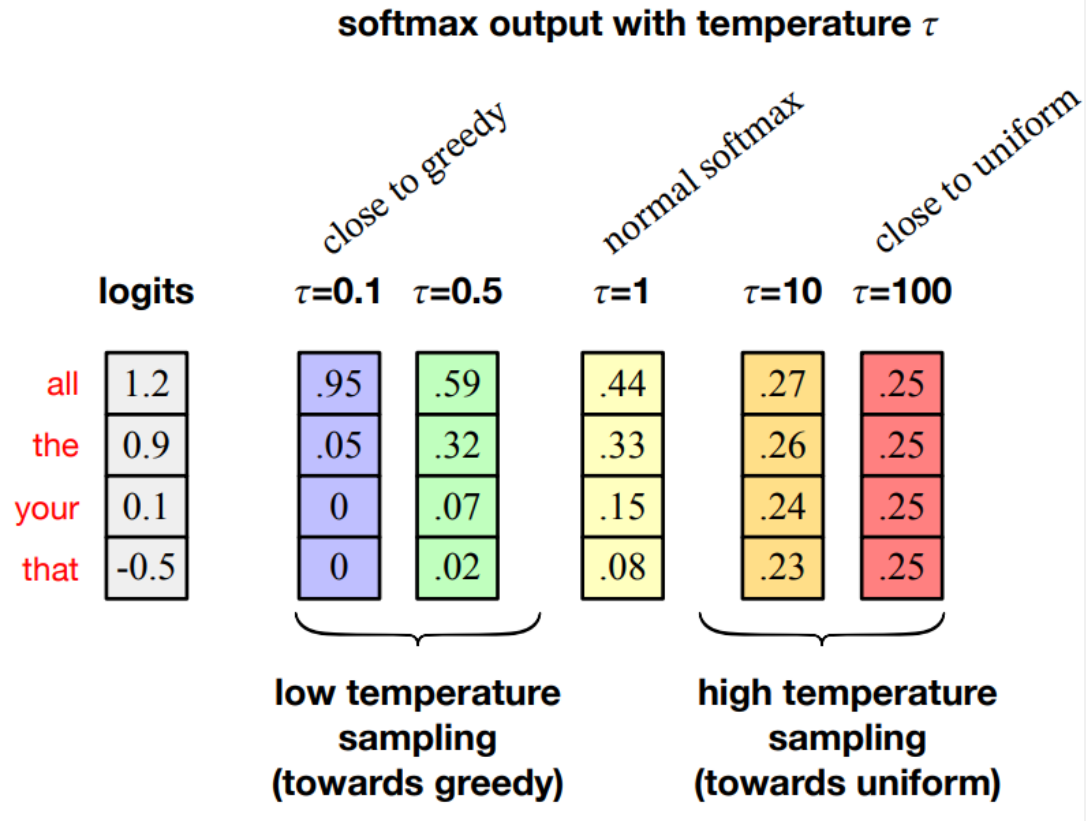
- El muestreo aleatorio puro presenta un **riesgo matemático grave** debido a la inmensidad del vocabulario de un modelo LLM (ej: 50k – 100k *tokens*).
- Generalmente, **sólo un puñado de *tokens* (ej: 2-5) son adecuados**, y son aquellos que acumulan la mayor parte de la probabilidad (ej. 70%).
- A pesar de ello, existen decenas de miles de palabras no tan adecuadas que, aunque cada uno tenga una probabilidad minúscula individual (ej. 0.001%), **la suma total de las probabilidades de todos ellos puede obtener un porcentaje representativo del total** (ej: 20-30%).
- Resultado: En el muestreo aleatorio puro, **existe un riesgo estadístico inaceptablemente alto de seleccionar un término no adecuado**, forzando al modelo a generar una palabra sin sentido que arruine el texto a generar. Este término **se añadirá al contexto, condicionando además la generación de los siguientes términos**.



# F3-MdL: El parámetro de temperatura (repaso)

- Para evitar este problema sin perder creatividad, modificamos la distribución de probabilidades:
  - Los modelos generan **logits** para cada término del vocabulario, que se transforman en probabilidades a través de la función softmax.
  - Para adaptar la distribución de probabilidades, dividimos cada logit por un parámetro denominado **temperatura** ($\tau$, $0 < \tau < \infty$) antes de aplicar softmax.
- Efecto del parámetro temperatura:
  - **Temperatura baja ($\tau < 1$) - "enfriar", menor creatividad**: Al dividir el logit por un decimal, los logits aumentan su valor absoluto. La función softmax exagera las diferencias entre las probabilidades de los términos: las probabilidades altas se elevan mucho y las bajas caen todavía más. El modelo se vuelve más estable, determinista y "*greedy*".
  - **Temperatura alta ($\tau > 1$) - "calentar", mayor creatividad**: Al dividir el logit por un número grande, los logits se acercan a cero. Como resultado, la diferencia entre las probabilidades de los términos se reduce, lo que aumenta la creatividad de las respuestas.

# F3-MdL: El parámetro de temperatura (repaso)



# F3-MdL: Filtrado de la distribución de probs. (repaso)

- Ajustar la temperatura nos permite modificar fácilmente la creatividad del modelo sin necesidad de reentrenarlo, pero no soluciona necesariamente el problema original (generación de texto no coherente).
- Para eliminar este problema, **debemos filtrar los términos no adecuados** antes del muestreo:
  - **Top-k *sampling***: Se ordenan los términos por probabilidad y se retienen únicamente las k opciones más probables (ej. k=40). El resto de términos ven su probabilidad forzada a 0%.
  - **Top-p *sampling* (*nucleus sampling*)**: En lugar de un número fijo de palabras, se establece un umbral de masa de probabilidad p (ej. p=0.90). Se van sumando las probabilidades de los términos más probables hasta alcanzar el umbral. El modelo realiza una selección aleatoria exclusivamente entre los términos de ese "núcleo" lógico, ignorando los demás.
- **Ventaja de top-p *sampling***: Se adapta dinámicamente. El núcleo podrá estar compuesto de un par de términos o de un centenar en función de si está muy seguro o no del término a predecir.

# F3-MdL: Condición de parada

- Los modelos Transformer generativos recurren a la generación autorregresiva para generar texto:
  - Recursivamente, generan un nuevo *token* en base a un contexto dado, añaden el *token* generado al contexto previo, y utilizan el nuevo contexto para generar un nuevo *token*.

- El vocabulario incluye etiquetas especiales como:
  - <BOS> (*begin of sequence*): Inicio de un texto o documento (entrenamiento).
  - <PAD> (*padding*): *Token* de relleno, sin valor semántico.
  - <UNK> (*unknown*): *Token* que no forma parte del vocabulario del modelo.
  - *Tokens* de rol (<|system|>, <|user|>, <|assistant|>).
  - ….
  - **<EOS> (*end of sequence*)**: Implica el fin de la generación.

- El modelo **aprende cuándo debe utilizar <EOS> durante el ajuste por instrucciones**. Durante este proceso, el modelo **aprende si ha satisfecho todas las tareas solicitadas, y bajo qué condiciones puede finalizar la generación**.

# F3-MdL: Penalización de repetición

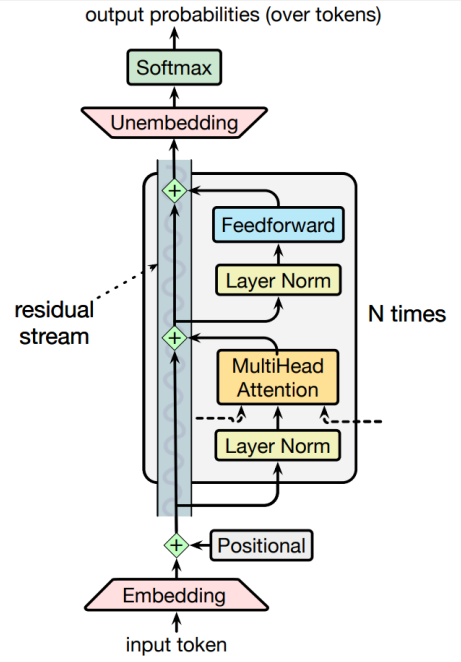
- Debido a la naturaleza de la arquitectura Transformer y el proceso de generación autorregresiva, un LLM puede entrar en un **bucle infinito** bajo el cual genera iterativamente el mismo *token*:
  - Cada *token* repetido hace que sea más probable generar de nuevo el mismo *token*.
- Solución: **Penalizar los *tokens* que se repiten dentro de la ventana de los últimos k *tokens***:
  - Una vez generados los logits de cada *token* del vocabulario, se reduce el valor de aquellos que se repiten en la ventana de últimos k *tokens*:
    - $Logit_i' = Logit_i - (freq_i \times rep_penalty) - (presence_i \times pres_penalty)$
    - Donde:
      - $freq_i$: Frecuencia de aparición del término i.
      - $rep_penalty$: Factor de penalización por repetición (hiperparámetro).
      - $presence_i$: Presencia del término i ($presence_i$ in {0,1}).
      - $pres_penalty$: Factor de penalización por presencia (hiperparámetro).

# 6

# Operación

# Operación: Resumen del ciclo de vida de un *token* en la arquitectura *decoder-only*

- **Fase 1: Encoding.**
  1. Tokenización.
  2. Cálculo de *embeddings* estáticos.
  3. (Opcional) Cálculo de *embeddings* posicionales y *embeddings* compuestos.
- **Fase 2: Bloques Transformer ($N_{layer}$ iteraciones).**
  1. Normalización.
  2. Atención multicabeza.
     - (Opcional) RoPE.
  3. Normalización.
  4. FNN.
     - (Opcional) MoE
- **Fase 3: Modelado de lenguaje / Decoding.**
  1. *Unembedding* (proyección a vocabulario, logits).
  2. Softmax (logits -> probabilidades).
     - Temperatura, filtrado.
  3. Selección del siguiente *token*.



# Operación: Flujo de ejecución

- Aunque la arquitectura del modelo es el mismo, el flujo de ejecución cambia en base a si estamos entrenando el modelo o infiriendo:
- **Durante el entrenamiento**:
  - Cogemos una **ventana de N *tokens***.
  - **Procesamos todos sus *embeddings* en paralelo**, mediante operaciones matriciales.
  - Aplicamos una **máscara de causalidad sobre la matriz de afinidad**, que lleve a 0 los pesos de atención de un *token* con *tokens* posteriores (no podemos entrenar conociendo el futuro).
  - Generamos, **para cada *token*, una predicción** sobre el siguiente *token*.

# Operación: Flujo de ejecución

- Aunque la arquitectura del modelo es el mismo, el flujo de ejecución cambia en base a si estamos entrenando el modelo o infiriendo:
- **Durante la inferencia. Dos fases**:
  - ***Prefill***:
    - **Procesamos todo el contexto disponible (ej: el *prompt*) en paralelo**, mediante operaciones matriciales.
    - Aplicamos una **máscara de causalidad** para **ocultar *tokens* posteriores**.
    - Durante este proceso:
      - **Se genera la predicción de siguiente *token* desde el último *token*** del contexto (primera palabra generada).
      - **Se construye la caché KV**, que permite generar nuevos *tokens* sin necesidad de calcular los *keys* y *values* de todos los *tokens* pasados.

# Operación: Flujo de ejecución

- Aunque la arquitectura del modelo es el mismo, el flujo de ejecución cambia en base a si estamos entrenando el modelo o infiriendo:
- **Durante la inferencia. Dos fases**:
  - ***Decode***:
    - **Generamos el 2$^{do}$ y siguientes *tokens***, mediante **generación autorregresiva**.
      - El contexto pasa a ser el contexto original + el 1$^{er}$ *token* generado.
    - No necesitamos procesar los *tokens* del pasado, **sólo el último generado**:
      - **Calculamos *query*, *key* y *value* para el último *token* generado**.
      - **Añadimos *key* y *value* a la caché KV**.
      - Llevamos a cabo el proceso de atención, **calculando los pesos de atención en base a la *query* y la matriz K** (de todos los *tokens* de contexto) y **transfiriendo contexto en base a la matriz V** (de todos los *tokens* de contexto).
- Si tras *decode* nos **interesa introducir nuevo contexto en el modelo (ej: 2$^{do}$ prompt)**, se realiza un procesado que **combina *prefill* con *decode***: calculamos los *query*, *key* y *value* de todos los nuevos *tokens*, actualizamos la caché KV, y llevamos a cabo el proceso de atención con las *querys* de todos los nuevos *tokens*, de manera simultánea.

# ¿Preguntas?